Ricci-Notation Tensor Framework for Numerical Algebraic Geometry

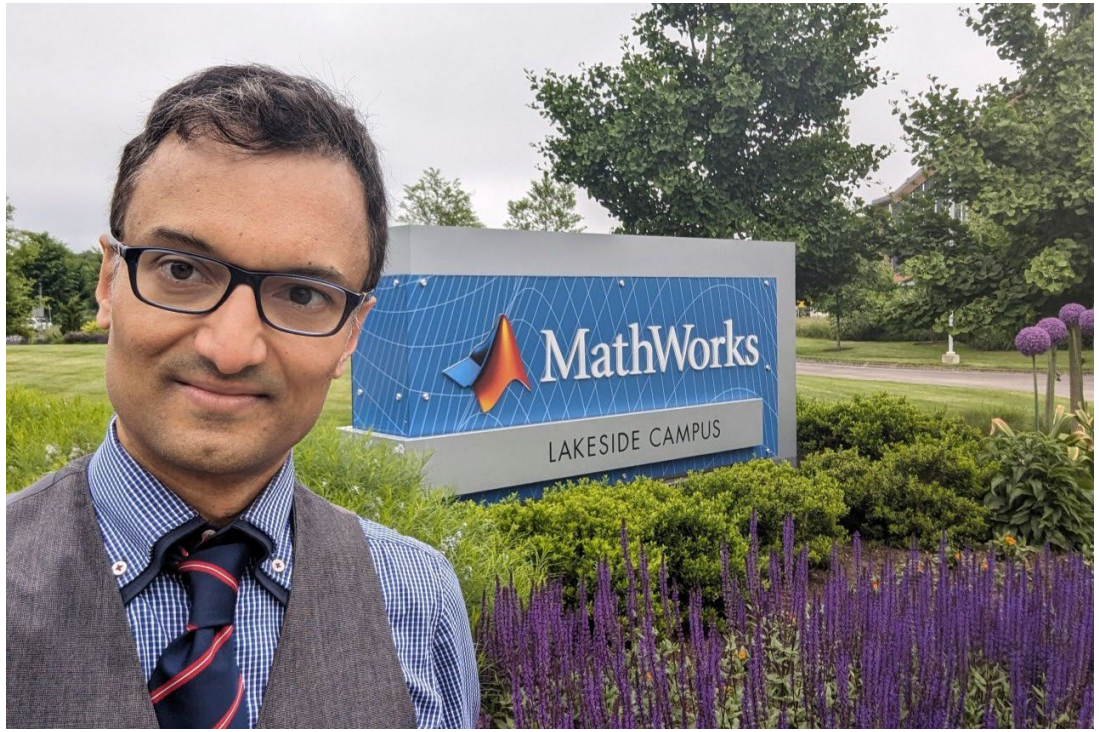
Dileepan Joseph, D.Phil. (Oxon), P.Eng. (AB)

Department of Electrical and Computer Engineering
University of Alberta, Edmonton, AB, Canada

June 22, 2026

Foreword

- This talk summarizes a 2026 paper, preprint in *arXiv*, on a Ricci-notation tensor (RT) framework extended to numerical algebraic geometry.
- The author thanks participants of the 2024 MathWorks Research Summit, at which he informally discussed the first RT framework, and others since for their encouraging feedback.
- For editorial advice on the paper and related materials, he especially thanks Dan Sirbu and Adam P. Harrison.



Joseph, Jun. 2024, at the MathWorks Lakeside Campus in Natick, MA, for an artificial intelligence (AI) summit.

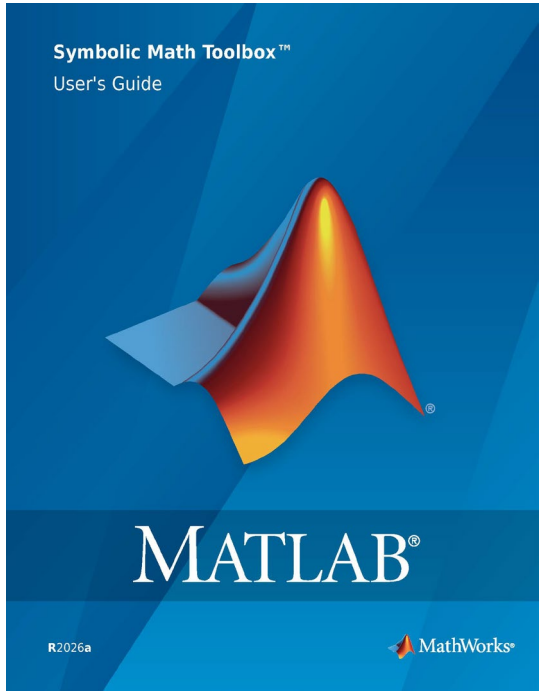
Introduction

- Algebraic geometry concerns the solution set, or zero set, of a system of polynomial equations. Application areas include modular robotics and cryptanalysis.
- A system may be positive dimensional (1^+D), where solutions include curved lines, surfaces, etc., or zero dimensional (0D), where only isolated points define solutions. For linear systems, this relates to rank.
- The preprint reviews algebraic geometry literature, categorizing solution methods into two *numerical* approaches, homotopy continuation and Macaulay canonical-polyadic-decomposition (Macaulay-CPD), and one *symbolic* approach, Gröbner basis (GB).

Human vs. robot in tic tac toe at the 2024 MathWorks Research Summit. Joseph won by switching from X to O, which confused the robot's AI.

Introduction

- While numerical methods for linear systems of equations, like Gauss elimination and unitary-triangular (QR) factorization, triangularize a given system, neither popular approach to numerical algebraic geometry does so.
- Related to Euclid's greatest-common-divisor algorithm and Gauss elimination, GB methods employ a triangularization-like ordering.
- One can substitute numbers into GB symbols. Due to triangularization and zero-set invariance, exactly one solution may be found, e.g., a global minimum, when all others are not needed.



The `gbasis` function of MATLAB's Symbolic Math Toolbox applies a GB algorithm with the option for a lexicographic monomial ordering. Cover taken from *User's Guide*, 2026.

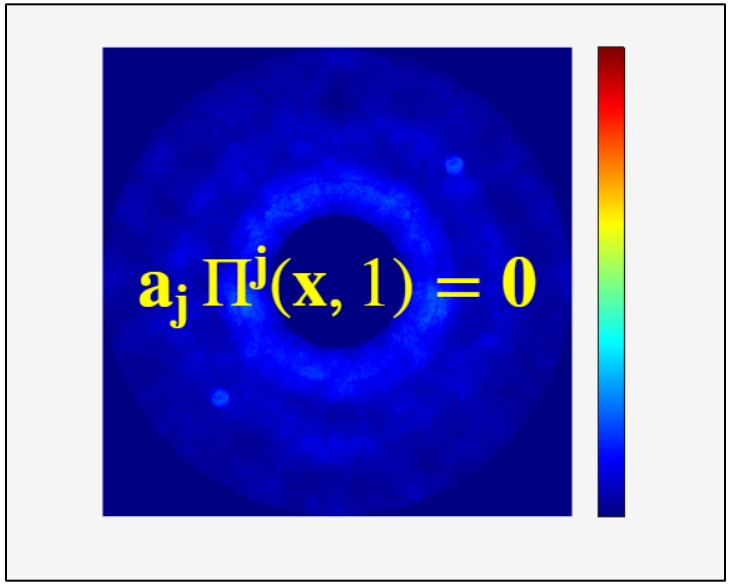
Introduction

- For 0D systems, the continuation and Macaulay-CPD approaches solve for all solutions. The latter employs a tensor model and factorization, the CPD, related to the null space of a *Macaulay* matrix. Solutions include zeros, not of the given system, that post-processing invalidates.
- CPD literature uses an *n*-mode notation, though tensors associate best with Ricci notation.
- For numeric purposes, Joseph has developed a Ricci-notation tensor (RT) framework. Via an *any-degree* unitary-triangular (Qr) factorization, this work extends it to algebraic geometry.

Joseph's initial RT framework appeared in *arXiv*, *MATLAB Central File Exchange*, and the *Journal of Imaging Science and Technology* (JIST). Cover taken from JIST, 2024.

Introduction

- The RT framework is the RT algebra and RTToolbox, MATLAB Central File Exchange, plus MATLAB.
- Proving zero-set invariance with Qr triangularization, this work generalizes the unitary transforms of linear algebra and linear systems to tensor algebra and polynomial systems, 0D and 1^{+}D, thereby extending the RT algebra, a dual-variant index notation.
- The work also updates the RTToolbox with: methods for algebraic geometry; a sparse multidimensional array (MDA) class; and a MATLAB executable (MEX) function to accelerate sparse operations. Simple concept demos illustrate developments.



Release 2026 (R2026) of the RTToolbox has a thumbnail that references a 2024 imaging example and a proposed RT model for a polynomial system.

Tensor Algebra

- Consider a canonical form opposite of a degree-D polynomial system, $\mathbf{a}\Pi$, with a vector-shaped coefficient tensor, $\mathbf{a}$, a vector of D indices, $\mathbf{j}$, a column vector of unknowns, $\mathbf{x}$, and a monomial-values function, Π.
- The function, Π, expresses the outer product of an augmented vector, $\mathbf{x}$, with itself after "reshaping":

$$\Pi^{\mathbf{j}}(\mathbf{x}, 1) = \Pi^{\mathbf{j}}\left(\begin{bmatrix}\mathbf{x}\\ 1\end{bmatrix}\right)$$
$$\Pi^{\mathbf{j}}(\mathbf{x}) = x^{j_1} x^{j_2} \dots x^{j_D}$$
$$x^j = (\mathbf{e}_j)^{\mathrm{T}}\mathbf{x}$$

Type	Univariate	Multivariate
Linear	$ax = b$ Scalar division	$\mathbf{Ax} = \mathbf{b}$ LU factorization, etc.
Polynomial	$a_n x^n \dots + a_1 x + a_0 = 0$ Companion matrix, etc.	$\mathbf{a}_{\mathbf{j}}\Pi^{\mathbf{j}}(\mathbf{x}, 1) = \mathbf{0}$ Qr factorization, etc.
Nonlinear	$f(x) = 0$ Brent's method, etc.	$\mathbf{f}(\mathbf{x}) = \mathbf{0}$ Newton's method, etc.

Adapted from Harrison's Ph.D. thesis, UAlberta, 2016.

- Consider also a homogeneous form and its QR factorization for a linear system:

$$[\mathbf{A} \quad -\mathbf{b}]\begin{bmatrix}\mathbf{x}\\ 1\end{bmatrix} = \begin{bmatrix}\mathbf{0}\\ 0\end{bmatrix}$$
$$[\mathbf{A} \quad -\mathbf{b}] = \mathbf{Q}[\mathbf{R} \quad -\mathbf{r}]$$

Tensor Algebra

- The proposed Qr factorization defines a degree-$(D + \Delta D)$ triangular system, $\mathbf{r}\Pi$, where system coefficients, $\mathbf{a}$ and $\mathbf{r}$, transform by two degree-ΔD unitary factors, one (conjugate) transposed:

$$\mathbf{r}_{\mathbf{k}} = (\mathbf{Q}_{\mathrm{u}}^{\mathbf{i}})^{*}\mathbf{a}_{\mathbf{j}}\varsigma_{\mathbf{k}}^{\mathbf{ij}}$$
$$\mathbf{a}_{\mathbf{i}}\varsigma_{\mathbf{j}}^{\mathbf{i}} = \mathbf{Q}_{\mathbf{h}}^{\mathrm{v}}\mathbf{r}_{\mathbf{k}}\varsigma_{\mathbf{j0}}^{\mathbf{hk}}$$

- A proposed variant sigma symbol, ς, resembles a “well known” Kronecker delta symbol, δ, of tensor calculus:

$$\varsigma_{\mathbf{j}}^{\mathbf{i}} = \begin{cases} 1, & \mathbf{j} = \mathrm{sort}(\mathbf{i}) \\ 0, & \text{otherwise} \end{cases}$$

- Forward and backward unitary factors, $\mathbf{Q}_{\mathrm{u}}$ and $\mathbf{Q}^{\mathrm{v}}$, must satisfy two identities:

$$(\mathbf{Q}_{\mathrm{u}}^{\mathbf{0}})^{*}\mathbf{Q}_{\mathrm{u}}^{\mathbf{0}} = \mathbf{I}$$
$$\mathbf{Q}_{\mathbf{h}}^{\mathrm{v}}(\mathbf{Q}_{\mathrm{u}}^{\mathbf{i}})^{*}\varsigma_{\mathbf{k}}^{\mathbf{hi}} = \mathbf{I}\delta_{\mathbf{k}}^{\mathbf{0}}$$

- A zero index vector, $\mathbf{0}$, means a vector of known end index values, similar to end as an MDA subscript in MATLAB.
- An upper-triangular constraint applies linearly to the triangular factor, $\mathbf{r}$, with summation of related coefficients:

$$\mathbf{r}_{\mathbf{k}} = \mathbf{e}_{m}\mathbf{e}_{n}^{\mathrm{T}}\mathbf{r}_{\mathbf{j}}\varsigma_{m\mathbf{k}}^{n\mathbf{j}}$$

Tensor Algebra

- Consider the zero set, $\mathcal{X}$, of a polynomial system, $\mathbf{a}\Pi$, having M equations and N unknowns. Each set member, $\mathbf{x} \in \mathcal{X}$, solves the system and no other solutions exist.
- Coefficients, $\mathbf{a}$, play a role in determining set size, related to rank in linear algebra. Simple relations, $M > N$, $M = N$, and $M < N$, do not define set size, whether empty, finite (i.e., 0D), or infinite (i.e., 1^{+}D).
- Two "Qr factorization" theorems claim that a zero of the given system implies a zero of the triangular system and vice versa.

Theorem 1. *Forward* invariance theorem.

The zero set of a *triangular* system, $\mathbf{r}\Pi$, contains the zero set of a *given* system, $\mathbf{a}\Pi$, where coefficients transform via a *forward* unitary *factor*, $\mathbf{Q}_\mathbf{u}$, obtained by Qr factorization of *given* coefficients, $\mathbf{a}$, i.e.:

$$\mathbf{r_k} = (\mathbf{Q}^{\mathbf{i}}_{\mathbf{u}})^{*}\mathbf{a_j}\varsigma^{\mathbf{ij}}_{\mathbf{k}}$$

Proof. The arXiv preprint details the proof.

The proof features the construction of a *forward* unitary *transform*, $\mathbf{U}$, that unlike in linear algebra incorporates the unknown variables, $\mathbf{x}$, i.e.:

$$\mathbf{U} = (\mathbf{Q}^{\mathbf{i}}_{\mathbf{u}})^{*}\Pi^{\mathbf{i}}(\mathbf{x}, 1)$$

With this matrix and via the RT algebra, the proof shows that the *triangular* system equals the *given* system premultiplied by the *forward* transform, $\mathbf{Ua}\Pi$, which guarantees *forward* invariance.

Tensor Algebra

- Whereas the Qr factorization adds a triangularity requirement, the theorem proofs depend only on properties of the unitary factors, $\mathbf{Q_u}$ and $\mathbf{Q^v}$, which define two unitary transforms, $\mathbf{U}$ and $\mathbf{V}$, that produce one system from another.
- Backward invariance requires only the second, two-factor, unitary identity and forward invariance requires neither.
- A solution to the two-factor identity may be transformed, round-off aside, into a solution that satisfies both identities.

Theorem 2. *Backward* invariance theorem.

The zero set of a *given* system, $\mathbf{a}\Pi$, contains the zero set of a *triangular* system, $\mathbf{r}\Pi$, where coefficients transform via a *backward* unitary *factor*, $\mathbf{Q^v}$, obtained by Qr factorization of *given* coefficients, $\mathbf{a}$, i.e.:

$$\mathbf{a_i}\varsigma_{\mathbf{j}}^{\mathbf{i}} = \mathbf{Q_h^v r_k}\varsigma_{\mathbf{j0}}^{\mathbf{hk}}$$

Proof. The arXiv preprint details the proof.

The proof features the construction of a *backward* unitary *transform*, $\mathbf{V}$, that unlike in linear algebra incorporates the unknown variables, $\mathbf{x}$, i.e.:

$$\mathbf{V} = \mathbf{Q_h^v}\Pi^{\mathbf{h}}(\mathbf{x}, 1)$$

With this matrix and via the RT algebra, the proof shows that the *given* system equals the *triangular* system premultiplied by the *backward* transform, $\mathbf{Vr}\Pi$, which guarantees *backward* invariance.

Tensor Algebra

- Algorithm 1 computes the Qr factorization of a real system, subject to parameters. As pseudocode, Algorithm 1 mixes RT algebra expressions with MATLAB-like syntax.
- A first part non-iteratively computes initial unitary factors using a weighted null-space basis to guarantee a triangular system.
- To satisfy both unitary identities, a second part defines and performs an iterative optimization. The last part non-iteratively completes the forward unitary factor and also computes the triangular factor.

Algorithm 1. Null-space method to obtain Qr factors.

$\mathbf{A}_{o\mathbf{k}}^{mn\mathbf{h}} \Leftarrow (\mathbf{e}_m)^{\mathrm{T}}\mathbf{a}_{\mathbf{i}}(\delta_o^n \varsigma_{\mathbf{k}}^{\mathbf{hi}} - \varsigma_{o\mathbf{k}}^{n\mathbf{hi}})$
$\mathbf{A} \Leftarrow \mathbf{e}_{mn\mathbf{h}}\mathbf{A}_{o\mathbf{k}}^{mn\mathbf{h}}(\mathbf{e}_{o\mathbf{k}})^{\mathrm{T}}$ % “zero”-cost reshape
$\mathbf{Z} \Leftarrow \mathrm{null}(\mathbf{A}^{\mathrm{T}})$ % i.e., where $\mathbf{A}^{\mathrm{T}}\mathbf{Z} \approx \mathbf{0}$
$\mathbf{Z}^{\mathbf{h}j} \Leftarrow \mathbf{e}_m((\mathbf{e}_{mn\mathbf{h}})^{\mathrm{T}}\mathbf{Z}\mathbf{e}^j)\mathbf{e}_n^{\mathrm{T}}$ % “zero”-cost reshape
$y_j \Leftarrow \mathbf{Z}^{\mathbf{0}j} \backslash \mathbf{I}$
$\mathbf{Q}_{\mathbf{h}}^{\mathrm{v}} \Leftarrow (\mathbf{I}\delta_{\mathbf{k}}^{\mathbf{0}})/((\mathbf{Z}^{\mathbf{i}j}y_j)^{\mathrm{T}}\varsigma_{\mathbf{k}}^{\mathbf{hi}})$

if options.MaxIterations > 0
    $\mathbf{E}_{\mathrm{u}} \Leftarrow @(y)\ (\mathbf{Z}^{\mathbf{0}j}y_j)^{\mathrm{T}}\mathbf{Z}^{\mathbf{0}j}y_j - \mathbf{I}$ % fwd. err.
    $\mathbf{E}_{\mathbf{k}}^{\mathrm{v}} \Leftarrow @(y, \mathbf{Q}^{\mathrm{v}})\ (\mathbf{Z}^{\mathbf{i}j}y_j)^{\mathrm{T}}\varsigma_{\mathbf{k}}^{\mathbf{hi}} - \mathbf{I}\delta_{\mathbf{k}}^{\mathbf{0}}$ % bwd. err.
    … % define sum squared error (SSE), f, etc.
    $[y, \mathbf{Q}^{\mathrm{v}}] \Leftarrow \mathrm{fminunc}(f, \nabla f, y, \mathbf{Q}^{\mathrm{v}}, \mathrm{options})$
end if

$\mathbf{Q}_{\mathrm{u}}^{\mathbf{i}} \Leftarrow \mathbf{Z}^{\mathbf{i}j}y_j$
$\mathbf{r}_{\mathbf{k}} \Leftarrow (\mathbf{Q}_{\mathrm{u}}^{\mathbf{i}})^{\mathrm{T}}\mathbf{a}_{\mathbf{j}}\varsigma_{\mathbf{k}}^{\mathbf{ij}}$

Tensor Algebra

- This work favours explainability for developing the Qr factorization, at this time, over detailed considerations of asymptotic complexity, i.e., scalability to very large polynomial systems.
- An outer-product model has benefits for explanation at the cost of index space proportional to $(N+1)^D$, e.g.:

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \\ 1 \end{bmatrix}^{\mathrm{T}} \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1N} & a_{10} \\ 0 & a_{22} & \cdots & a_{2N} & a_{20} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & a_{NN} & a_{N0} \\ 0 & 0 & \cdots & 0 & a_{00} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \\ 1 \end{bmatrix} = 0$$

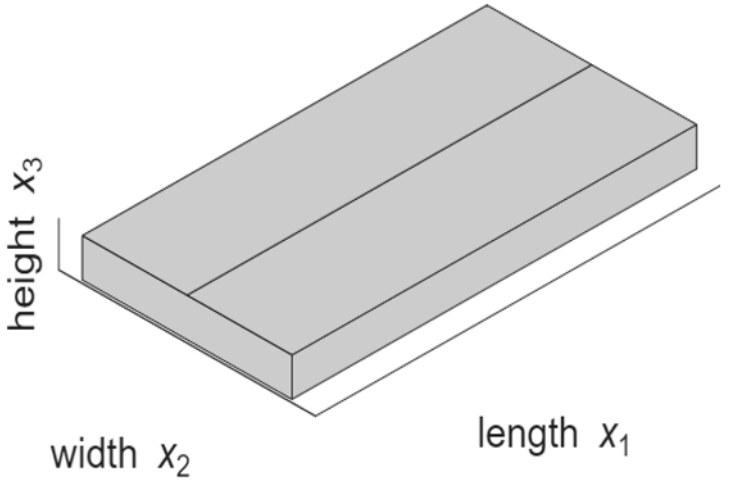
- The related Qr software internally switches from an outer-product to a unique-monomial model and back, which eliminates null-space basis redundancies in Algorithm 1.
- At the cost of some bookkeeping, a unique-monomial model has index space proportional to $(D+1)^N$, e.g.:

$$\begin{bmatrix} a_{11} & \cdots & a_{10} & a_{22} & \cdots & a_{00} \end{bmatrix} \begin{bmatrix} x_1x_1 \\ \vdots \\ x_1 \\ x_2x_2 \\ \vdots \\ 1 \end{bmatrix} = 0$$

Concept Demos

- Given the total linear dimension, $1\ \mathrm{m}$, the total surface area, $0.5\ \mathrm{m}^2$, and the volume, $0.01\ \mathrm{m}^3$, of a box, a system of three polynomial equations with three unknowns specifies all dimensions.
- With the RT software, R2026, a sparse `tensor` object defines the system.

```
>> a = boxdims(1,0.5,0.01);
>> disp(a)
   ...
   (2,1,2,3,4)      2
   (2,1,4,4,4)   -0.5000
   (3,1,1,2,3)      1
   (3,1,4,4,4)   -0.0100
```



- The given system, expressed via the RT algebra in the canonical form:

$$\underbrace{\begin{bmatrix} x_1 + x_2 + x_3 - 1 \\ 2x_1x_2 + 2x_1x_3 + 2x_2x_3 - 0.5 \\ x_1x_2x_3 - 0.01 \end{bmatrix}}_{\mathbf{a}_j \Pi^j(\mathbf{x},1)} = \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}}_{\mathbf{0}}$$

Concept Demos

- A `qr` method, given ΔD, performs Qr factorization based on Algorithm 1.

```
>> [Qu,r] = qr(a,2,'Display','iter');
Initial point is a local minimum.
>> iseq = @(a,b,n) isequal( ...
     round(a,n),round(b,n));
>> [vs,~,k] = vsigma(Qu',a);
>> assert(iseq(r(k),Qu'*a*vs,15))
```

- The Qr triangularization happens to equal a reduced GB system here:

$$\underbrace{\begin{bmatrix} x_1 + x_2 + x_3 - 1 \\ -2x_2^2 - 2x_2x_3 + 2x_2 - 2x_3^2 + 2x_3 - 0.5 \\ x_3^3 - x_3^2 + 0.25x_3 - 0.01 \end{bmatrix}}_{\mathbf{r_k \Pi^k(x,1)}} = \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}}_{\mathbf{0}}$$

- A `roots` method solves a univariate equation using MATLAB's `roots`.

```
>> k = index(r);
>> x3 = roots(r(end,1,k));
```

- The three roots, 0.6264, 0.3244, and 0.0492 m, provide a zero vector of the original system due to symmetry, as a `polyval` method can demonstrate.

```
>> x = sort(x3,'descend');
>> polyval(a,x,1)
ans =
   1.0e-15 *
    0.2220
    0.2220
    0.0035
```

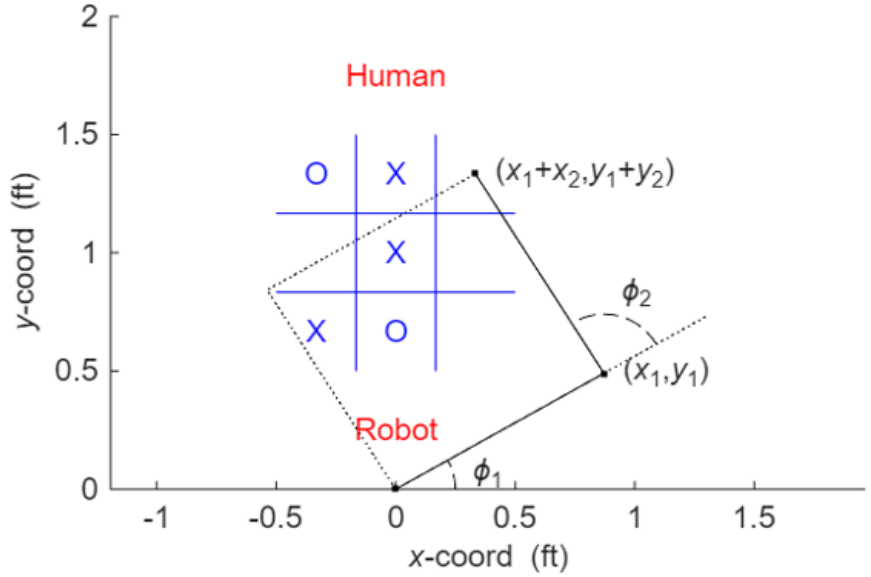
$$\mathbf{a_j \Pi^j(x,1) = 0}$$

Concept Demos

- A game-playing robot has an arm with two 1 ft segments and one end at the origin. To direct the other to a point, $(1/3\,, 4/3)$ ft, yields a system over four variables, x_1, y_1, x_2, and y_2, in ft:

$$\underbrace{\begin{bmatrix} x_1 + x_2 - 1/3 \\ y_1 + y_2 - 4/3 \\ x_1^2 + y_1^2 - 1 \\ x_2^2 + y_2^2 - 1 \end{bmatrix}}_{\mathbf{a_j}\Pi^{\mathbf{j}}(\mathbf{w},1)} = \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}}_{\mathbf{0}} \qquad \begin{aligned} \phi_1 &= \operatorname{atan2}(y_1, x_1) \\ \phi_2 &= \operatorname{atan2}(y_2, x_2) \\ \\ \mathbf{w}^{\mathrm{T}} &= [y_2\ x_2\ y_1\ x_1] \end{aligned}$$

- Algorithm 1 requires many iterations to satisfy both unitary identities.



- The qr method can output additional tensors, $\mathbf{Q}^{\mathrm{V}}$ and $\mathbf{I}\delta$, for identity tests.

```
>> a = arm2seg(1/3,4/3);
>> [Qv,Qu,r,Id] = qr(a,2);
```

Concept Demos

```
>> I = speye(size(Qu,2);
>> assert(isequal(Id(:,:,end),I))
>> assert(isequal(nnz(Id),nnz(I)))
>> Qu0 = Qu(:,:,end);
>> assert(iseq(Qu0'*Qu0,I,3))
>> [vs,~,k] = vsigma(Qv,Qu');
>> assert(iseq(Qv*Qu'*vs,Id(k),2)
```

- How Algorithm 1 performs depends in part on compilation of a supplied MEX function, spmex1, and its use cases.

```
>> % mex -R2018a -Drrlu spmex1
>> % mex -R2018a -Dbsx spmex1
>> mex -R2018a spmex1
```

- Proving too slow, the quasi-Newton optimization was easy to explain.

Qr concept demo: GB method (gbasis):	boxdims 0.01 (s)		arm2seg 0.01 (s)	
% mex -R2018a spmex1	3.93	4.01	9.14	29.7
mex … -Drrlu spmex1	3.85	3.92	9.13	21.6
mex … -Dbsx spmex1	0.03	0.05	0.03	19.7
mex -R2018a spmex1	0.03	0.04	0.03	15.1
options.MaxIterations	0	400	0	400

- Without spmex1 or with -Drrlu, qr does not use MATLAB's null function. But it can be tested, proving slow like without spmex1 or with -Drrlu.

```
>> r = qr(a,2,@(A) null(A,'rational'));
```

Concept Demos

- With the arm2seg demo, after zeroing near-zero coefficients, the univariate equation yields accurate x_1 roots.

```
>> a = arm2seg(1/3,4/3);
>> r = qr(a,2); % 400 iterations
>> r(abs(r) < 1e-15) = 0;
>> x1Qr = roots(r(end,1,index(r));
>> x1GB = roots([68/9 -68/27 -287/81]);
>> assert(iseq(x1Qr,x1GB,15))
```

- Toward a sparser Qr triangularization, the arm2seg function offers sparser unitary factors, derived by hand.

```
>> [a,Qu,Qv,Id] = arm2seg(1/3,4/3);
>> [Qu,Qv,R] = qq(Qu,Qv); % Transform
>> r = Qu'*a*vsigma(Qu',a);
```

- The Qr triangularization obtained this way has 18 nonzeros, down from 118 with a default 400 iterations. It equals a nonsingular upper-triangular matrix, $\mathbf{R}$, times a reduced GB equivalent:

$$\underbrace{\mathbf{R}\begin{bmatrix} 8/3\,y_2 - 2/3\,x_1 - 5/3 \\ x_2 + x_1 - 1/3 \\ 8/3\,y_1 + 2/3\,x_1 - 17/9 \\ 68/9\,x_1^2 - 68/27\,x_1 - 287/81 \end{bmatrix}}_{\mathbf{r_k \Pi^k(w,1)}} = \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}}_{\mathbf{0}}$$

- Thus, the arm2seg demo supplements the RTToolbox, R2026, with additional resources to improve on Algorithm 1.

Tensor Software

- With MATLAB and the RTToolbox, `tensor` and `index` objects may be constructed and used consistent with the RT algebra, as extended.
- A `sparse1` class and an `spmex1` executable, two of multiple software developments, help to express and accelerate *sparse* tensor algorithms that leverage dual-variant index notation.
- Although MATLAB has dense MDAs, no built-in sparse MDA type exists.

M- or C-file	Summary	Note
`tensor.m`	Definition of `tensor` objects	Revised
`tensorTest.m`	Tests of `tensor` objects	(R2024)
`index.m`	Definition of `index` objects	(R2024)
`indexTest.m`	Tests of `index` objects	(R2024)
`page*.m`	Pagewise helper functions	(R2024)
`qrssefg.m`	Required for `fminunc` use	Added
`sparse1.m`	Definition of `sparse1` objects	Added
`sparse1Test.m`	Tests of `sparse1` objects	Added
`spmex1.c`	Accelerate `sparse1` methods	Added
`spmex1Test.m`	Tests of `spmex1` executable	Added
`kdelta.m`	Kronecker delta (δ) sparse tensor	Added
`vsigma.m`	Variant sigma (ς) sparse tensor	Added
`boxdims*.m`	Box dimensions concept demo	Added
`arm2seg*.m`	Robotic arm concept demo	Added

Tensor Software

- The `sparse1` class uses MATLAB's sparse 1D arrays, sometimes 2D, to define sparse MDA objects, with reduction of all-zero columns in the 2D case for asymptotic efficiency.
- The `sparse1` constructor follows the behaviour of MATLAB's `sparse` function. A new four-argument form can construct a sparse MDA.
- Methods accelerated directly, like `rref`, have `try-catch` blocks. The unary `null` method calls `rref`.

Public methods of `sparse1` class (*`spmex1` acceleration)
Constructor, *etc.*
`sparse1` (1–6 args in), `find`, `disp`, `full`, `sparse`, `logical`, `double`, `ndims`, `size`, `end`, `numel`, `isempty`, `length`, `nnz`, `nzmax`, `issparse`, `isreal`, `class`, `isa`, `subsindex`
Unary operations
`subsref`, `permute`, `reshape`, `sum`, `max`, `min`, `all`, `any`, `abs`, `uminus`, `uplus`, `conj`, `sqrt`, `log`, `not`, `real`, `imag`, `round`, `pagetranspose`, `pagectranspose`, `pagetrace`, `pagediag`, `transpose`, `ctranspose`, `trace`, `diag`, `rref*`, `null*`

- If `spmex1` or a use case, like `'rrlu'`, is missing, the `catch` block uses MATLAB functions, like its `rref`, to do the job.

Tensor Software

- Binary `plus` to `complex` methods have `try-catch` paths. As `spmex1` has a relevant use case, `'bsx'`, for real `double` arrays only, the `eq` to `ge` methods create `A-B` first.
- Block-diagonal `mtimes`, `mldivide`, and `mrdivide` of sparse 2D arrays can compute the `pagemtimes` and `pagem?divide` of sparse MDAs.
- For `mtimes` and `m?divide`, excising in coordination all-zero rows and columns improves efficiency.

Public methods of `sparse1` class (*`spmex1` acceleration)
Binary operations
`subsasgn`, `plus*`, `minus*`, `eq*`, `ne*`, `lt*`, `gt*`, `le*`, `ge*`, `and`, `or`, `times*`, `ldivide*`, `rdivide*`, `power`, `complex`, `pagemtimes`, `pagemldivide`, `pagemrdivide`, `mtimes`, `mldivide`, `mrdivide`
N-ary operations
`isequal`, `cat`, `horzcat`, `vertcat`

- Complexity of *N*-ary methods depends linearly on the sum, given all `sparse1` operands, of the numbers of nonzeros, due mainly to sparse 1D array design.

Tensor Software

- A mid-level approach, `spmex1` exploits a C MEX programming interface to compute a null-space precursor, a row-reduced echelon form (RREF), via rank-revealing LU (RRLU) factorization.
- For scaling purposes, compute-bound binary-singleton-expansion (BSX) operations generalize Khatri-Rao products and replace memory-bound alternatives that use MATLAB unary-singleton-expansion (USX) functions, `repmat` or `repelem`, on operands.

Use cases of `spmex1` executable

`spmex1('debug',A)` displays MEX details of a MATLAB sparse `double` array, which must have size `[M N]`.

`[L,U,p,q] = spmex1('rrlu',A)` returns a sparse LU factorization with full pivoting, where `L*U` approximates `A(p,q)`, that also reveals rank. The main diagonal of `L` is all one and the main diagonal of `U` is all nonzero up to the rank, `r`, of `A`. All rows of `U` below row `r` are zero.

`C = spmex1('bsx',fun,A,B)` returns an operation, `fun`, applied outerwise over rows and entrywise over columns of the operands. The output, `C`, has size `[M*N P]` if the inputs, `A` and `B`, have sizes `[M P]` and `[N P]`.

- Currently, the design supports only real arrays of built-in type `double`.

Tensor Software

- With no dependencies outside of the MEX interface and standard C library, `spmex1` has three subroutine groups.
- The main subroutine `mexFunction` decides the use case of `spmex1` from its arguments and dispatches control to subroutine pairs, like `mx*RRLU`.
- Subroutine `mxCreateRRLU` employs directly or indirectly most of the compute and rearrange subroutines. Minimizing memory reallocation, it uses C pointers and `for` loops.

Subroutines of `spmex1` executable
Main, *etc.*
`mexFunction`, `mxCheckDebug`, `mxPrintDebug`, `mxCheckRRLU`, `mxCreateRRLU`, `mxCheckBSX`, `mxCreateBSX`
Compute
`mxGetPivot`, `mxSetLCol`, `mxSetDeltaA`, `mxUseDeltaA`, `mxFinishL`, `mxPivotSeq`, `mxBSXPlus`, `mxBSXTimes`
Rearrange
`mxMoveRow`, `mxMoveCol`, `mxMoveVal`, `mxSetNzmin`

- Compute subroutines `mxBSXPlus` and `mxBSXTimes` perform plus- and times-like, not for `Infs` or `NaNs`, operations.

Conclusions

- Two main approaches to *numerical* algebraic geometry solve, without creating a triangular system, for all solutions to a polynomial system.
- *Continuation* does not concern tensor notation and/or algorithms, whereas *Macaulay-CPD* uses *n*-mode notation to develop a tensor decomposition.
- Unlike the *symbolic* GB approach, which does produce a triangular-like system, the CPD approach computes vectors that must be invalidated.
- A proposed Qr factorization defines a triangularization that ensures zero-set invariance, while extending a tensor algebra built on the *Ricci* notation.
- The RT algebra generalizes unitary transforms to the algebraic geometry, possibly 1^{+}D curved spaces, defined by polynomial-system zero sets.
- A preliminary Qr algorithm employs a null-space basis that, however, is not the null space of a Macaulay matrix but one that ensures triangularity.

Conclusions

- More important than an iterative part of the proposed algorithm, this work contributed the RTToolbox, R2026, to help develop improved algorithms.
- R2026 introduces: `tensor` methods for algebraic geometry with RT `index` notation; and a sparse MDA class plus MEX function, `sparse1` and `spmex1`.
- The latter represent and manipulate sparse MDAs using 1D and sometimes compressed 2D sparse MATLAB arrays, with selected operations done in C.
- Two simple problems, `boxdims` and `arm2seg`, demonstrate key concepts of the proposed RT framework for numerical algebraic geometry.
- For `boxdims`, Qr factorization yields without iteration a GB-like result. For `arm2seg`, a handcrafted sparser result proves the Qr concept's soundness.
- The work demonstrates acceleration of key steps via `spmex1`, like sparse null-space calculation, in comparison to using MATLAB's own functions.